Exhaustive Manual Testing Guide: Stock Management (Stocks, Stock Ledgers, Stock Adjustments, Stock Transactions, Stock Valuations, Stock Reservations)

1. Overview & Architecture Scope

The Stock Management module constitutes the core physical and financial inventory accounting engine of the Textile ERP system. It governs real-time stock balances across physical branches and warehouses, polymorphic transaction logs, manual stock write-offs/adjustments, immutable daily stock ledger rolling balances, inventory asset valuation engines, and temporary order stock reservations.

Target Components & Resources:

- **Current Stocks:** Stocks (Product, Variant, Branch, Warehouse, Quantity, Weighted Avg Cost)
- **Transaction Logs:** Stock Transactions (Stock Linkage, Transaction Type: in/out/adjustment/transfer, Quantity, Polymorphic Reference ID & Type)
- **Stock Adjustments:** Stock Adjustments (Branch, Warehouse, Product, Quantity, Reason, Status: pending/approved/rejected)
- **Stock Ledgers:** Stock Ledgers (Product, Variant, Branch, Date, Opening Qty, Inward Qty, Outward Qty, Closing Qty)
- **Stock Valuations:** Stock Valuations (Branch, Valuation Date, Total Inventory Asset Value)
- **Stock Reservations:** Stock Reservations (Product, Variant, Branch, Quantity, Reference Order ID, Status: active/fulfilled/cancelled)

---

2. Pre-requisites & Test Environment Setup

1. Database migrated cleanly (`php artisan migrate:fresh`).
2. Super Admin user created with full WMS permissions.
3. Master reference entities initialized:
    - **Active Branch & Warehouse:** `Coimbatore Central Branch` & `Central Distribution Warehouse`.
    - **Base Product & Variant:** `Royal Cotton Formal Shirt (Red – Size 40)`.

---

3. Comprehensive Test Cases

3.1 Current Stock Master (`Stocks`)

Test Case 1.1: Stock Record Initialization & Location Mapping

- **Objective:** Verify initial creation of stock records per product/variant, branch, and warehouse.
- **Pre-conditions:** Logged in as Super Admin.
- **Action:** Create new Product Variant `Royal Cotton Shirt (Red/40)`. Check `stocks` table.
- **Expected Result:** Stock record initialized for `Coimbatore Branch` with `quantity = 0.00` and `avg_cost = 0.00`.

Test Case 1.2: Moving Weighted Average Cost (`avg_cost`) Calculation Engine

- **Objective:** Verify weighted average cost recalculation on incoming inventory batches.
- **Action 1 (Batch 1 Inward):** Receive 100 Pcs @ ₹100.00/unit via GRN.
    - **Expected Result:** Quantity = `100.00`, Total Cost = ₹10,000.00, `avg_cost = 100.00`.
- **Action 2 (Batch 2 Inward):** Receive 100 Pcs @ ₹150.00/unit via GRN.
    - **Expected Computation:** New Total Qty = `200.00 Pcs`, Total Inventory Value = ₹10,000 + ₹15,000 = ₹25,000.
    - **Expected Result:** System automatically recalculates `avg_cost` = ₹25,000 / 200 = `125.00 INR`.

Test Case 1.3: Multi-Branch & Multi-Warehouse Isolation

- **Objective:** Ensure stock quantities are strictly isolated by branch and warehouse.
- **Action:** Add 50 Pcs stock to `Chennai Branch`. Query stock for `Coimbatore Branch`.
- **Expected Result:** `Coimbatore Branch` stock remains `200.00 Pcs` without data leakage.

3.2 Stock Transactions Audit Log (`StockTransactions`)

Test Case 2.1: Transaction Record Generation (`in`, `out`, `transfer`, `adjustment`)

- **Objective:** Verify every inventory movement generates a polymorphic audit transaction.
- **Action:** Perform GRN inward (+100), POS sale (-5), Stock Transfer (-20), and Stock Adjustment (-2).
- **Expected Result:** `stock_transactions` table logs 4 distinct rows with correct `transaction_type` tags and exact quantities.

Test Case 2.2: Polymorphic Reference Linkage Verification

- **Objective:** Ensure transactions link cleanly to source documents.
- **Action:** Inspect `stock_transactions` table for GRN inward entry.
- **Expected Result:** `reference_type` stores `App\Models\Grn` and `reference_id` stores exact GRN document ID.

Test Case 2.3: Immutability Audit Guardrail

- **Objective:** Prevent manual tampering of raw transaction logs.
- **Action:** Attempt to manually edit `stock_transactions` row quantity via UI/API.
- **Expected Result:** Operation blocked: "Stock transaction logs are immutable audit records and cannot be directly modified."

3.3 Stock Adjustments (`StockAdjustments`)

Test Case 3.1: Stock Adjustment Approval Workflow (Draft -> Approved)

- **Objective:** Manage manual stock write-offs with multi-tier approval.
- **Action 1 (Create Request):** Go to `Stock Adjustments -> Create`. Select Product `Royal Cotton Shirt (Red/40)`, Quantity `-10.00`, Reason `Water Damage during transit`. Status = `pending`. Save.
    - **Expected Result:** Adjustment logged in `pending` status. Stock remains unchanged at `200.00 Pcs`.
- **Action 2 (Supervisor Approval):** Log in as Store Supervisor -> Click `Approve`.
    - **Expected Result 1:** Status changes to `approved`.
    - **Expected Result 2:** Current `Stocks.quantity` decreases from `200.00` to `190.00 Pcs`.
    - **Expected Result 3:** New `stock_transactions` row created with type `adjustment` and qty `-10.00`.

Test Case 3.2: Rejected Adjustment Isolation

- **Objective:** Ensure rejected adjustments do not alter physical stock.
- **Action:** Create adjustment for `-50.00 Pcs`. Supervisor clicks `Reject`.
- **Expected Result:** Status updates to `rejected`. Physical stock remains unchanged at `190.00 Pcs`.

Test Case 3.3: Negative Stock Adjustment Guardrail

- **Objective:** Prevent adjustments from reducing stock below zero when negative stock is disabled.
- **Action:** Create negative adjustment for `-300.00 Pcs` (current stock is `190.00 Pcs`). Approve.
- **Expected Result:** System blocks approval: "Stock adjustment rejected. Total physical stock cannot be negative (Available: 190.00 Pcs)."

3.4 Stock Ledgers (`StockLedgers`)

Test Case 4.1: Daily Stock Ledger Equilibrium Formula

- **Objective:** Verify immutable daily stock balance calculations.
- **Formula:** `closing_qty = opening_qty + inward_qty - outward_qty`
- **Action:** For Date `2026-08-01`:
    - Opening Qty = `100.00`
    - Inward Qty (GRN) = `50.00`

    - Outward Qty (POS Sale) = `20.00`

- **Expected Result:** `StockLedgers.closing_qty` MUST calculate to `130.00 Pcs`.

Test Case 4.2: Automated End-of-Day (EOD) Ledger Rollover

- **Objective:** Verify previous day's closing stock becomes next day's opening stock.
- **Action:** Trigger EOD midnight cron job for `2026-08-02`.
- **Expected Result:** System automatically creates ledger entry for `2026-08-02` setting `opening_qty = 130.00 Pcs` (matching `2026-08-01` closing stock).

3.5 Stock Valuations (`StockValuations`)

Test Case 5.1: Periodic Branch Inventory Asset Valuation Run

- **Objective:** Calculate total financial inventory asset valuation per branch.
- **Action:** Navigate to `Stock Valuations -> Run Valuation` for `Coimbatore Branch`.
- **Computation:** Sum of (`Stocks.quantity * Stocks.avg_cost`) across all active variants in branch.

    - Item A: 190 Pcs @ ₹125.00 = ₹23,750.00
    - Item B: 50 Pcs @ ₹500.00 = ₹25,000.00

- **Expected Result:** Total Valuation record saved with `valuation_date = 2026-08-01` and `total_value = 48,750.00 INR`.

Test Case 5.2: Financial General Ledger Equilibrium Check

- **Objective:** Verify total stock valuation matches Inventory Asset account in Finance module.
- **Action:** Compare `StockValuations.total_value` with `Chart of Accounts: 1100 - Merchandise Inventory`.
- **Expected Result:** Asset values match perfectly.

3.6 Stock Reservations (`StockReservations`)

Test Case 6.1: Temporary Order Stock Reservation (Sales Order Placement)

- **Objective:** Reserve stock for pending customer orders to prevent double-selling.
- **Action:** Create Sales Order `#SO-9901` for `15.00 Pcs` of `Royal Cotton Shirt (Red/40)`.
- **Expected Result 1:** Record logged in `stock_reservations` with `quantity = 15.00`, `status = active`.
- **Expected Result 2 (Available Stock Check):**

    - Total Physical Stock = `190.00 Pcs`.
    - Reserved Stock = `15.00 Pcs`.
    - Net Available Stock for new POS sales = `175.00 Pcs`.

Test Case 6.2: Reservation Fulfillment on Billing / Dispatch

- **Objective:** Fulfill reservation upon invoice completion.
- **Action:** Process invoice billing for `#SO-9901`.
- **Expected Result:** Reservation status updates to `fulfilled`. Physical stock decrements by `15.00 Pcs` (New stock: `175.00 Pcs`). Active reservation clears.

Test Case 6.3: Reservation Release on Order Cancellation

- **Objective:** Release reserved stock back to available pool upon order cancellation.
- **Action:** Cancel active Sales Order `#SO-9902` (reserved `10.00 Pcs`).
- **Expected Result:** Reservation status updates to `cancelled`. Net available stock increases by `+10.00 Pcs`.

---

4. Pass / Fail Verification Checklist Matrix

Module Component	Test Scenario	Expected Result	Status
Current Stocks	Weighted Avg Cost Calculation	`avg_cost` recalculates on new batch inward	Pass
Stock Transactions	Polymorphic Audit Logs	Logs `in/out/transfer/adjustment` with reference IDs	Pass
Stock Adjustments	Approval Workflow & Deduction	Approval reduces physical stock; Rejection leaves stock intact	Pass
Stock Ledgers	Daily Equilibrium Formula	`closing_qty = opening + inward - outward` holds true	Pass
Stock Valuations	Total Inventory Asset Run	Valuation matches aggregate DB math & GL account	Pass
Stock Reservations	Temporary Order Hold & Release	Active reservation holds stock; Cancellation releases stock	Pass